



AuditWare

X

0xbow

Smart Contract Security Review

Performed by **Auditware** for **0xbow**



auditware.io



audit_wizard



Review Summary

The Auditware team has performed a security review on the 0xbow's Privacy Pools protocol's Batch Relayer implementation for privacy pool batch withdrawal functionality.

The scope consisted of the following contracts:

- **packages/contracts/src/contracts/BatchRelayer.sol** - Main contract implementation
- **packages/contracts/src/interfaces/IBatchRelayer.sol** - Interface definition
- **packages/contracts/script/BatchRelayer.s.sol** - Deployment script

Audit commit hash: [2f6a650](#)

Although best practices were applied and the contract was written with security in mind, several issues were identified during the audit. Detailed explanations per item can be found in the report.

As with all smart contracts, it is highly recommended to:

- Conduct regular and thorough audits to identify and mitigate vulnerabilities.
- Achieve the highest possible test coverage, ideally approaching 100%.
- Maintain comprehensive and up-to-date documentation.
- Participate in auditing contests and/or a bug bounty program.
- Implement on-chain monitoring for real-time threat detection and mitigation.



Contracts Overview

The BatchRelayer contract enables atomic batch withdrawals from privacy pools, allowing users to process multiple withdrawals in a single transaction while maintaining privacy guarantees. The implementation follows a stateless architecture that accepts pool and processor addresses as parameters, differing from the existing Entrypoint contract's registry-based scope validation approach. The BatchRelayer only allows withdrawals, simplifying the flows and validation requirements of the standard Entrypoint.

Security Architecture

Core Security Controls:

- Reentrancy Protection: OpenZeppelin ReentrancyGuard prevents recursive calls during batch processing
- Cryptographic Integrity: ZK proof verification with context binding prevents proof manipulation
- Input Validation: Comprehensive validation of batch parameters, fee limits, and withdrawal amounts
- Safe Asset Handling: OpenZeppelin SafeERC20 and proper native asset transfer patterns

Batch Processing Flow:

- Users generate ZK proofs binding withdrawals to specific processor and pool addresses
- BatchRelayer validates batch parameters and total value calculations
- Each withdrawal proof is verified against the bound processor context
- PrivacyPool contracts validate that the calling processor matches the proof-bound processor
- Funds are transferred atomically with fee deduction and recipient distribution
- Architecture Comparison: Unlike Entrypoint's dual-layer validation (registry lookup + processor self-validation), BatchRelayer relies on direct parameter passing with validation primarily occurring in the PrivacyPool contracts through msg.sender checks.



Threat Model

Malicious pool substitution

Description: Attackers could deploy pool contracts with identical interfaces but malicious logic. Users may be tricked into generating proofs against them via phishing or compromised UIs.

Analysis: Entrypoint contracts fetch pool addresses from an admin-controlled registry, ensuring only pre-approved pools can be used. BatchRelayer allows arbitrary pool definitions, which could expand the attack surface if a malicious pool definition were combined with UI compromise.

Risk: High

Malicious relayer / BatchRelayer substitution

Description: A compromised or malicious relayer could substitute an attacker-controlled BatchRelayer contract, enabling theft of withdrawn funds.

Analysis: Relayer authenticity is not validated, consistent with Entrypoint design. Withdrawals are integrity-protected by proofs, but without canonical relayer validation a malicious relayer paired with a successful UI compromise could redirect withdrawn funds.

Risk: High

Relayer processing incomplete batches

Description: Relayers could drop or skip some withdrawals to disrupt user batch withdrawal amounts.

Analysis: The contract enforces that the proof array size must equal the withdrawal count, otherwise the call reverts. The `context` signal in proofs provides additional authenticity.

Risk: Medium

Entrypoint vs. BatchRelayer trust model divergence

Description: Differences in validation logic between the two contracts could create gaps in protection.

Analysis: Entrypoint validates processors via pool-level checks, while BatchRelayer omits some validation steps. Likely because user funds can only be deposited to authentic pools via the Entrypoint, mitigating the effective risk of a malicious BatchRelayer pool usage. This divergence means the two contracts rely on different assumptions, which could result in differing logic



flows between the two contracts.

Risk: Medium

BatchRelayData overflow into `totalValue`

Description: Possibility that a large batchSize could overflow into totalValue and alter withdrawal amounts.

Analysis: Overflow is potentially theoretically possible, but would only invalidate the proof and cause a revert. `totalValue` is not used for calculations beyond a consistency check, so practical impact is minimal.

Risk: Low

Relayer denial-of-service

Description: Malicious relayers could refuse to process transactions or censor withdrawals.

Analysis: This risk exists but is consistent with current design assumptions. No additional code paths beyond Entrypoint were found that increase exposure.

Risk: Low

Findings Overview

Finding	Component	Type	Risk Level
<u>AW-H-01: Unrestricted Processor Address Acceptance</u>	Batch Relayer	Loss of Funds	High
<u>AW-M-01: Unvalidated Pool Parameter</u>	Batch Relayer	Insufficient Validation	Medium
<u>AW-L-01: Missing Constructor Parameter Validation</u>	Batch Relayer	Insufficient Validation	Low
<u>AW-L-02: Batch Size Type Limitation</u>	Batch Relayer Interface	Insufficient Validation	Low



AW-H-01: Unrestricted Processor Address Acceptance

Severity: *High*

Status: *Unmitigated*

Code:

- [packages/contracts/src/contracts/BatchRelayer.sol#L26](#)

Description:

During the audit it was found that users can be socially engineered into generating proofs with malicious processor addresses, and the protocol has no way to distinguish between legitimate and malicious processors.

The current validation in PrivacyPool.sol:

```
// Check caller is the allowed processoor
if (msg.sender != _withdrawal.processoor) revert
InvalidProcessoor();

// Check the context matches to ensure its integrity
if (_proof.context() !=
uint256(keccak256(abi.encode(_withdrawal, SCOPE))) %
Constants.SNARK_SCALAR_FIELD) {
    revert ContextMismatch();
}
```

This only ensures that:

- The caller matches the processor specified in the withdrawal
- The proof was generated for this specific withdrawal data

But still enables attackers to deploy malicious processors with identical interfaces and steal all withdrawn funds.

The attack requires minimal technical sophistication only contract deployment and social engineering with no gas costs or technical barriers once users are deceived.



An attacker abusing this scenario might follow these steps:

- Attacker deploys malicious BatchRelayer contract with identical interface
- Through compromised UI, social engineering, or phishing, tricks user into generating withdrawal proofs on a valid pool with malicious processor address
- User unknowingly authorizes withdrawal to attacker's contract instead of legitimate BatchRelayer
- Malicious contract receives all withdrawn funds and redirects them to attacker instead of intended recipient
- Attack scales if malicious processor address spreads through compromised UIs or documentation

Recommendations:

- Processor Whitelist: Add a whitelist of approved processor addresses at the PrivacyPool level
- Circuit-Level Validation: Include processor address validation within the zero-knowledge circuit itself
- Registry Pattern: Implement a canonical processor registry that users/UIs can reference



AW-M-01: Unvalidated Pool Parameter

Severity: **Medium**

Status: **Unmitigated**

Code:

- [packages/contracts/src/contracts/BatchRelayer.sol#L27](#)

Description:

BatchRelayer accepts any pool address as a parameter without validation, contrasting with Entrypoint's registry-based approach that provides cryptographic assurance of pool legitimacy. This architectural difference creates conditional fund loss scenarios where users interact with unverified pools:

```
// Entrypoint validates pools via registry:
IPrivacyPool _pool = scopeToPool[_scope];
if (address(_pool) == address(0)) revert PoolNotFound();

// BatchRelayer accepts any pool address directly
```

The risk manifests when attackers deploy malicious pool contracts implementing IPrivacyPool interfaces with dishonest logic. Unlike Entrypoint's technical enforcement through registry validation, BatchRelayer relies entirely on user behavior and UI integrity for pool verification, creating a trust gap that sophisticated attackers can exploit through social engineering campaigns or compromised UIs.

An attacker may abuse this by following the next scenario:

- Attacker deploys malicious pool contract implementing **IPrivacyPool** interface with malicious logic
- Through social engineering, phishing sites, or compromised documentation, tricks users into believing malicious pool is legitimate
- User generates a withdrawal proof against the malicious pool
- User receives fake ERC-20 tokens that they would assume are legitimate.
- Unlike Entrypoint's registry protection, BatchRelayer provides no technical barrier to prevent interaction with malicious pools



Recommendations:

- Implement comprehensive pool validation. You may follow one of these suggestions:
 - Integrating with Entrypoint's existing pool registry to validate **_pool** addresses against **scopeToPool** mapping
 - Creating a dedicated pool registry with administrative controls for adding verified pools. As an interim measure, clearly document pool verification requirements in user-facing interfaces and provide canonical pool addresses through official channels to reduce social engineering attack surface.



AW-L-01: Missing Constructor Parameter Validation

Severity: **Low**

Status: **Unmitigated**

Code:

- [packages/contracts/src/contracts/BatchRelayer.sol#L19](#)

Description:

The BatchRelayer constructor accepts any value for **MAX_RELAY_FEE_BPS** without bounds validation, creating deployment-time configuration risk.

```
constructor(uint256 _maxRelayFeeBPS) {  
    MAX_RELAY_FEE_BPS = _maxRelayFeeBPS;  
}
```

While the current deployment script correctly sets 1000 (10%), the lack of validation could theoretically allow deployment with fees exceeding 100%.

This creates operational risk if deployment scripts are modified incorrectly or if the contract is deployed manually without proper parameter verification. This can pose a risk of future deployments could inadvertently create unusable contracts with impossible fee structures.

Recommendations:

- Add comprehensive constructor validation to prevent misconfiguration:

```
constructor(uint256 _maxRelayFeeBPS) {  
    require(_maxRelayFeeBPS <= 10_000, "Fee exceeds 100%");  
    require(_maxRelayFeeBPS > 0, "Fee must be positive");  
    MAX_RELAY_FEE_BPS = _maxRelayFeeBPS;  
}
```

This ensures deployment-time parameter validation and provides clear error messages for misconfiguration attempts.



AW-L-02: Batch Size Type Limitation

Severity: **Low**

Status: **Unmitigated**

Code:

- [packages/contracts/src/interfaces/IBatchRelayer.sol#L24](#)

Description:

During the audit, it was found that the **BatchRelayData** struct uses **uint8** for batch size, which limits batch operations to a maximum of 255 withdrawals:

```
struct BatchRelayData {  
    // ...  
    uint8 batchSize;      // Maximum 255 batches  
    uint256 totalValue;  
}
```

This type constraint prevents processing larger batch operations that might be required by power users, institutional withdrawals, or automated systems processing high volumes.

While not a direct security risk, the limitation could impact user experience and protocol adoption for large-scale operations. Users requiring >255 withdrawals must execute multiple batch transactions.

Recommendations:

- Consider **uint16** for batch size if larger batches are anticipated, or document the 255-withdrawal limit.





This report was produced by Auditware for Oxbow based solely on the information provided by Oxbow. Auditware provides no guarantees as to the accuracy of the contents of this report and does not make any guarantees that following the advice within will prevent security incidents or issues.

Auditware is available for questions or comments about any of the contents of this report. We can be reached at <https://auditware.io/> or by email at joe@auditware.io.

